Linear Regression

ICT 3.3 Machine learning and Data Mining

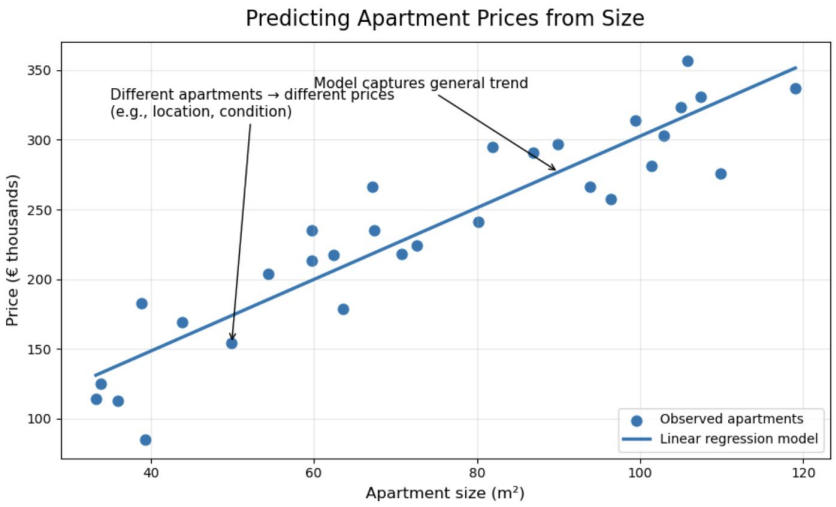
Emanuela Boros, May 2026

Why Linear Regression?

This is one of the most important, interpretable, and simplest ML models

Let's take a simple real-world problem: predicting the price of a new apartment on the market.

We could try to look at similar apartments… but instead, we want a general rule.



Linear regression assumes a simple relationship:
price ≈ w × size + b

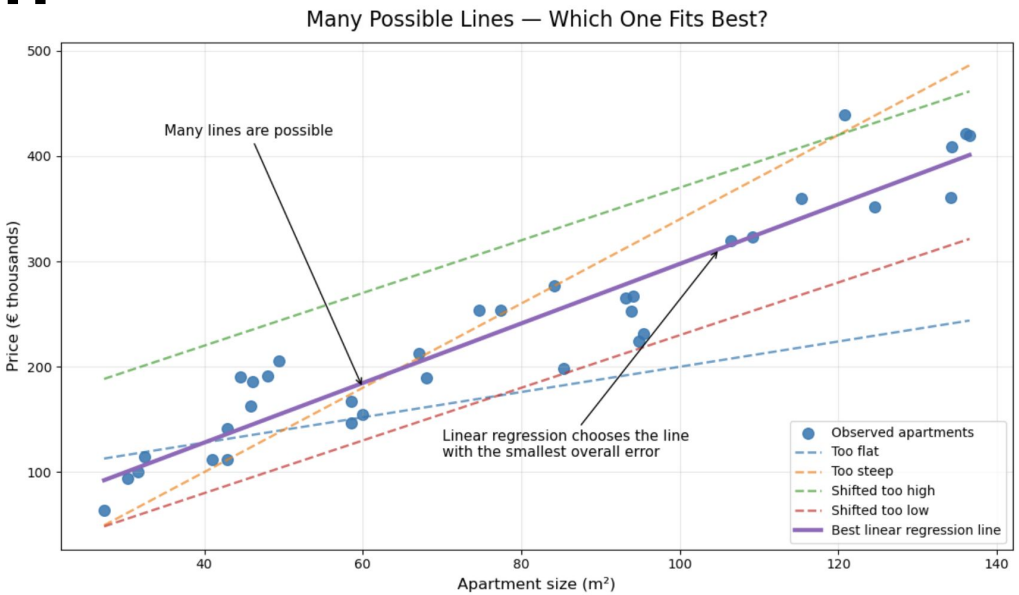
Linear Regression

We assume a simple relationship: a line

The problem is finding the right position of this line in order to separate the data..

$$z = wx + b$$

- x: input (e.g. size of apartment)
- y: output (price)
- w: slope (weight)
- b: bias



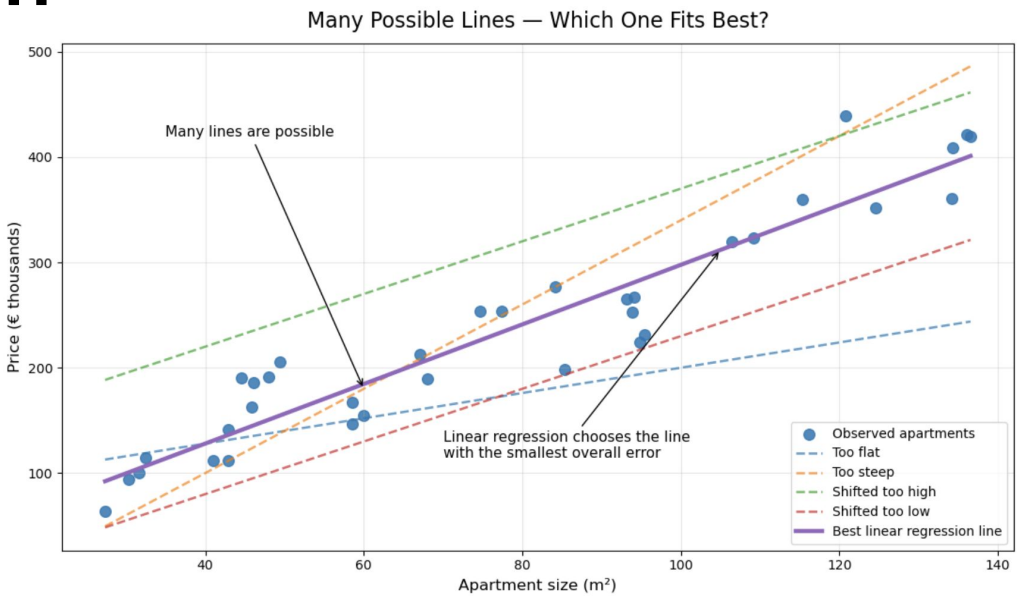
- w → controls the slope
- b → shifts the line up/down

Linear Regression

$$z = wx + b$$

Prediction — *Weight* — Feature value — *Bias*

Calculated during training



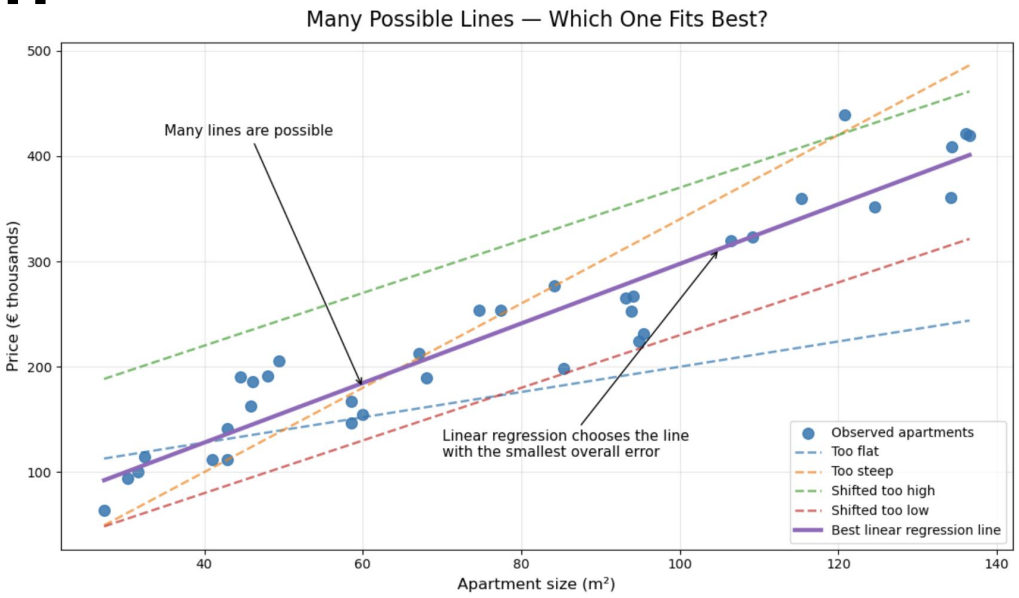
- w → controls the slope
- b → shifts the line up/down

Linear Regression

$$z = wx + b$$

Price — *Weight* — Size — *Bias*

Calculated during training

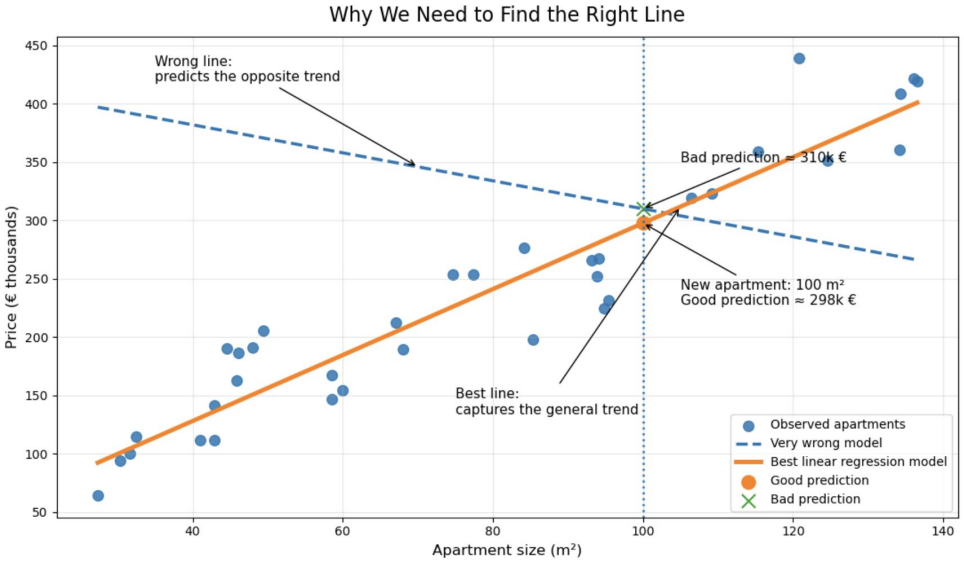


- w → controls the slope
- b → shifts the line up/down

Linear Regression

How do we find the best line automatically?
We need a way to measure:

- how good a line is
- how far its predictions are from the real data

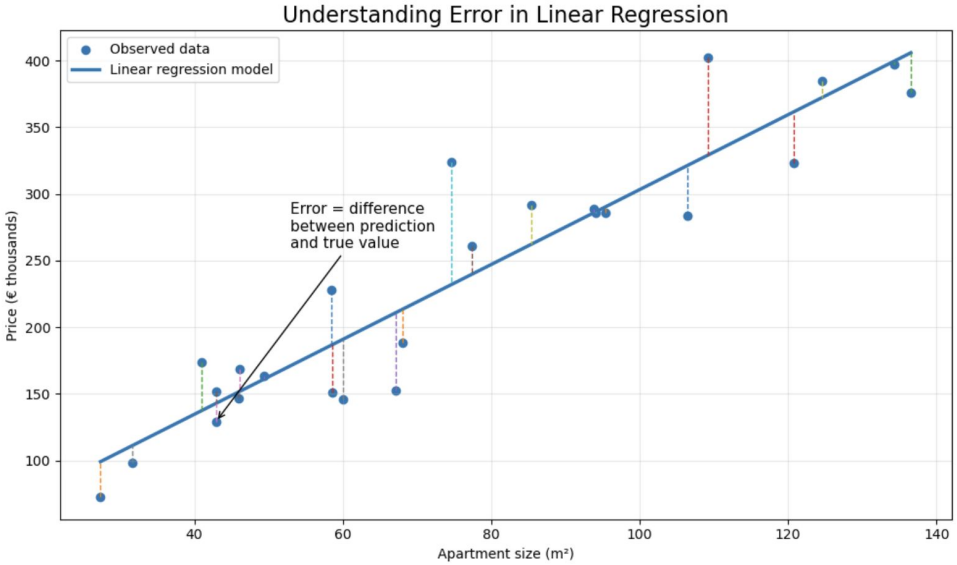


The best line is the one that makes the smallest overall prediction error.

Measuring Error: Mean Squared Error (MSE)

The vertical distance = “error” (residual)

→ difference between predicted value and actual value $y_i - \hat{y}_i$



If we have many points… how do we measure total error?

Measuring Error: Mean Squared Error (MSE)

→ average and we square them to avoid cancellation and penalize large errors

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE measures how wrong the model is. If this number is large… what should we do?

Measuring Error: Mean Squared Error (MSE)

→ average and we square them to avoid cancellation and penalize large errors

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE measures how wrong the model is. If this number is large… what should we do?

We want to lower it.

Measuring Error: Mean Squared Error (MSE)

→ average and we square them to avoid cancellation and penalize large errors

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE measures how wrong the model is. If this number is large… what should we do?

We want to lower it.

Training = reducing this error

Training = finding the parameters (weights) that minimize the loss

Gradient descent

Gradient descent is an optimization method used to find the model parameters — the weights and bias → that produce the lowest loss.

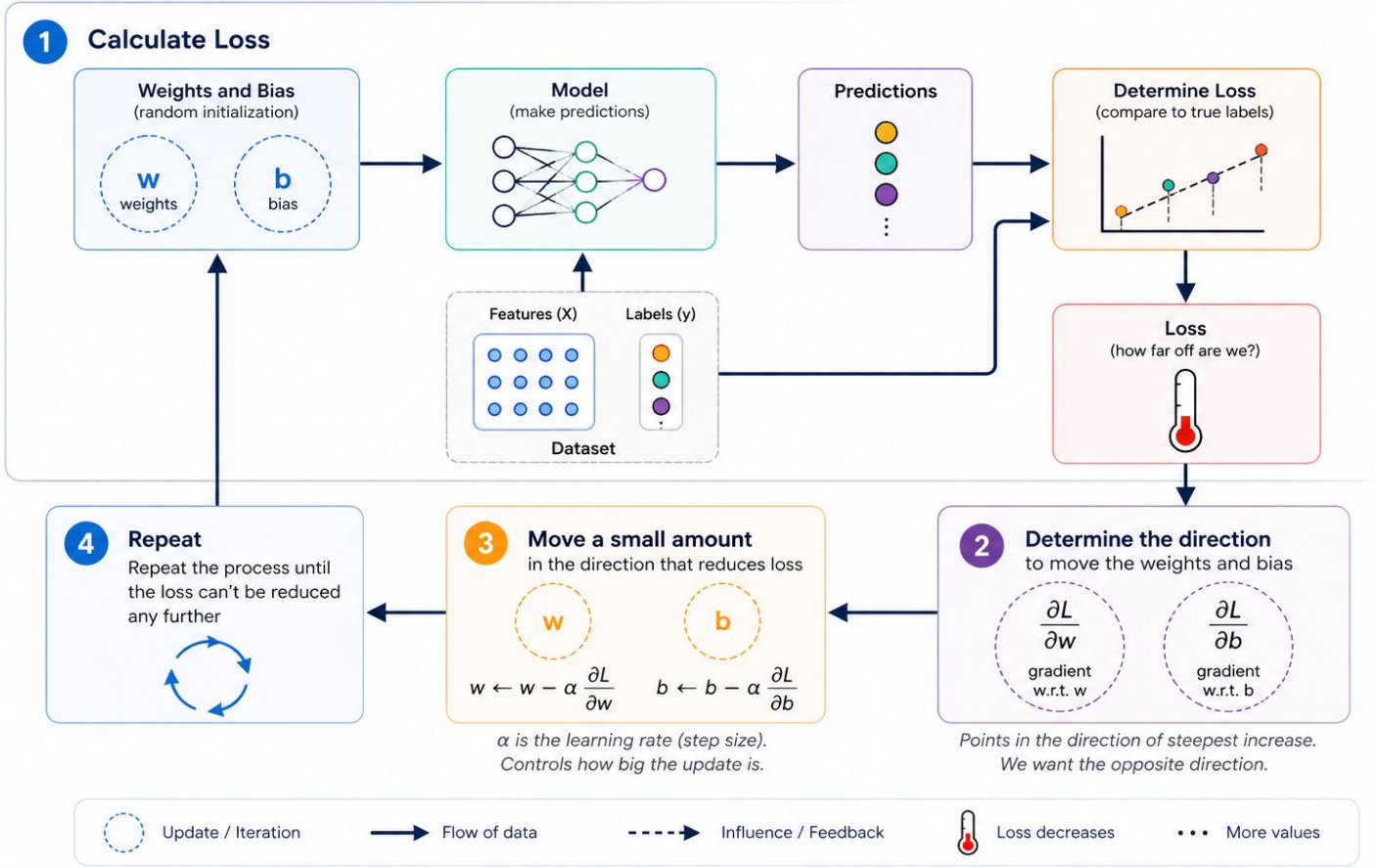
The model starts with random values for the weights and bias, then improves them step by step.

At each iteration, it:

1. Computes the current loss
2. Finds the direction that would reduce the loss
3. Updates the weights and bias slightly in that direction
4. Repeats until the loss stops improving

GRADIENT DESCENT

Iteratively find the weights (w) and bias (b) that minimize the loss



Training by Optimization - How do we learn w?

weights

scale the step with learning rate

the gradient → the partial derivative of L with respect to w

move against the gradient

We minimize the loss $\min_{w} \mathcal{L}(w)$

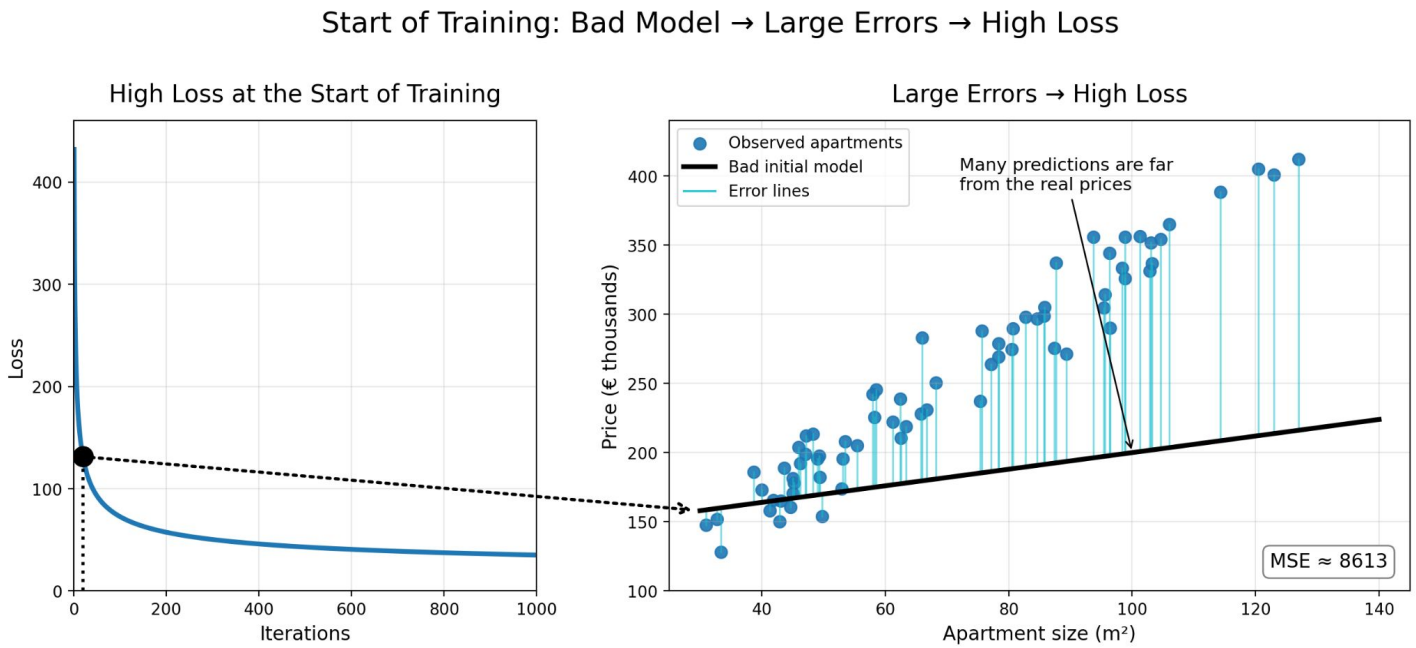
Gradient descent: $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$

We start with random weights then we iteratively update them to reduce the loss.

gradient = direction of increase → we go in opposite direction towards decrease!

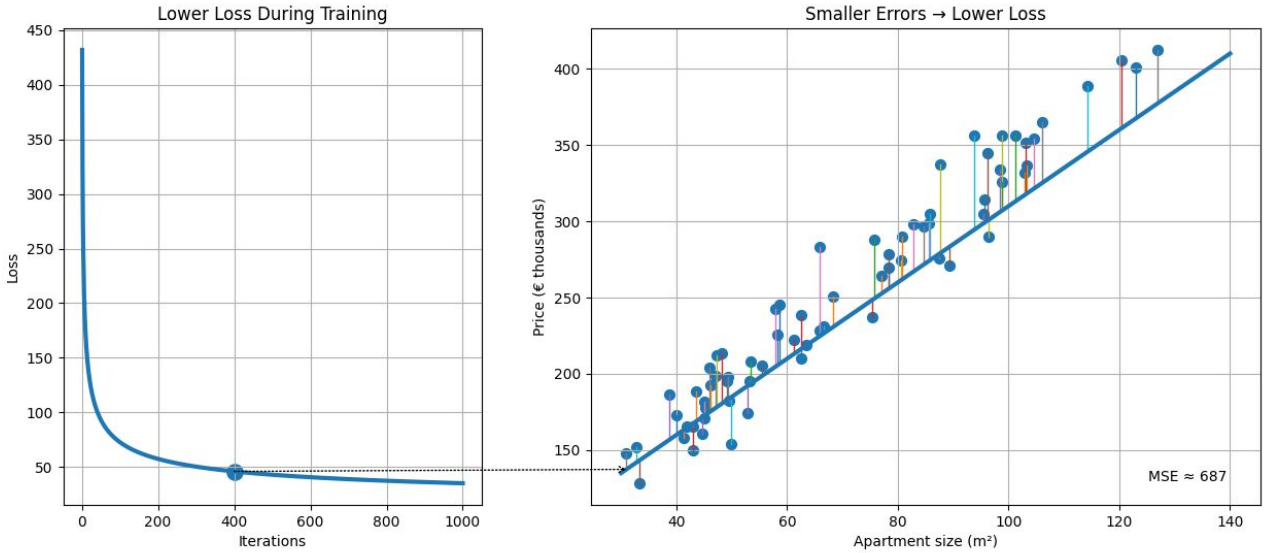
We practically move **w** step by step to reduce error.

Start of training



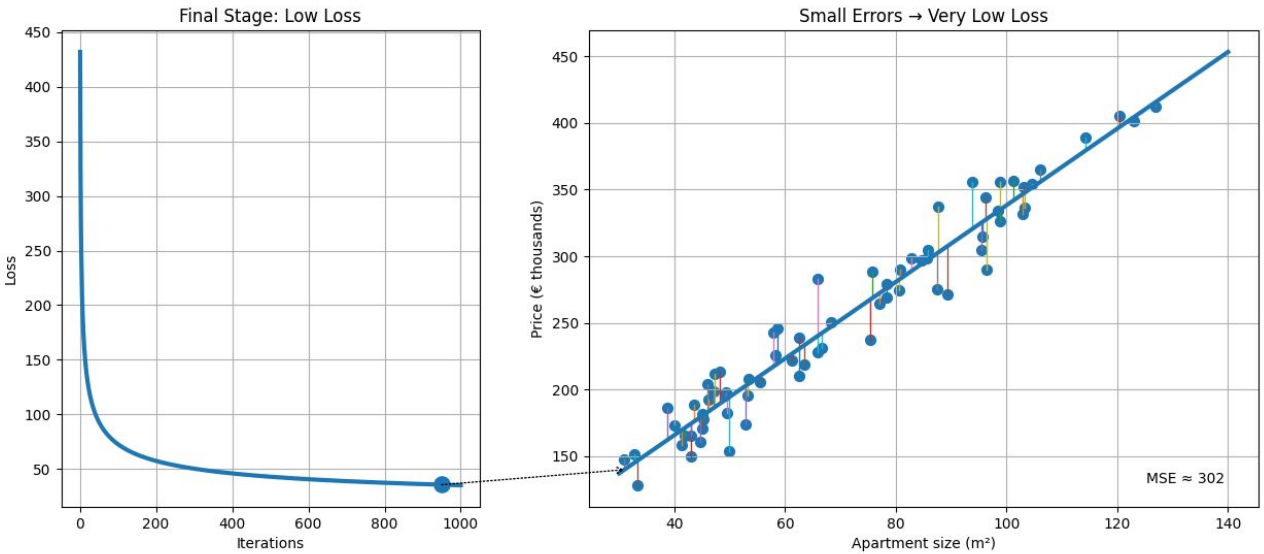
- Loss is high
- The model line is far from the data (poor fit)

During training



- Loss is decreasing
- The model line is closer to the data but still not good enough

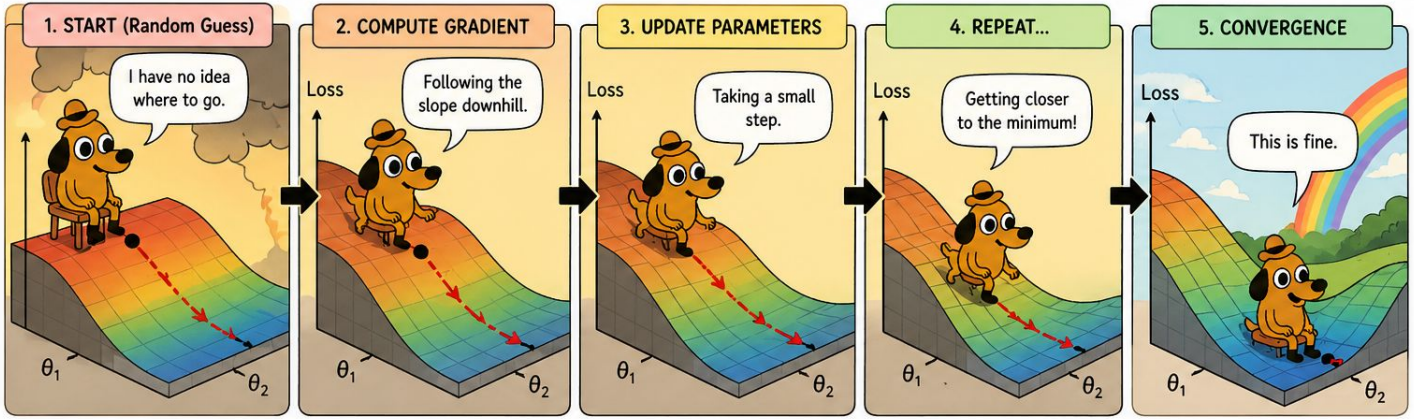
After training



- Loss is low and almost flat
- The model line fits the data well (small errors)

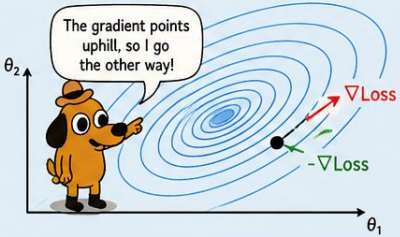
GRADIENT DESCENT: FINDING THE LOWEST LOSS

Goal: Find the parameters (θ) that minimize the loss.



HOW IT WORKS

1. Start with random parameters (θ).
2. Compute the gradient (slope) of the loss.
3. Update parameters in the opposite direction of the gradient.
4. Repeat until the loss is (almost) minimized.



RESULT

Gradient Descent finds the set of parameters that gives the lowest possible loss (local minimum).

Learning rate (α) controls the step size.

Too big? You might overshoot.

Too small? It takes forever.

● = current position

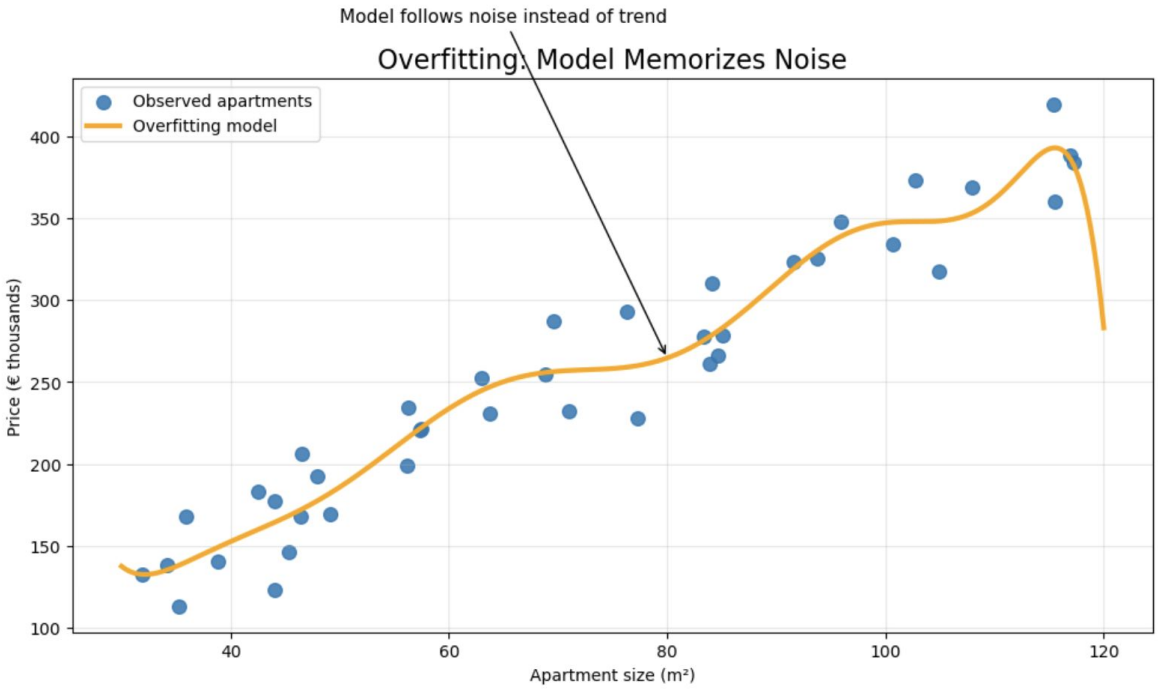
→ = path

ML Key challenges

- underfit → too simple
- overfit → too complex
- good fit → generalizes

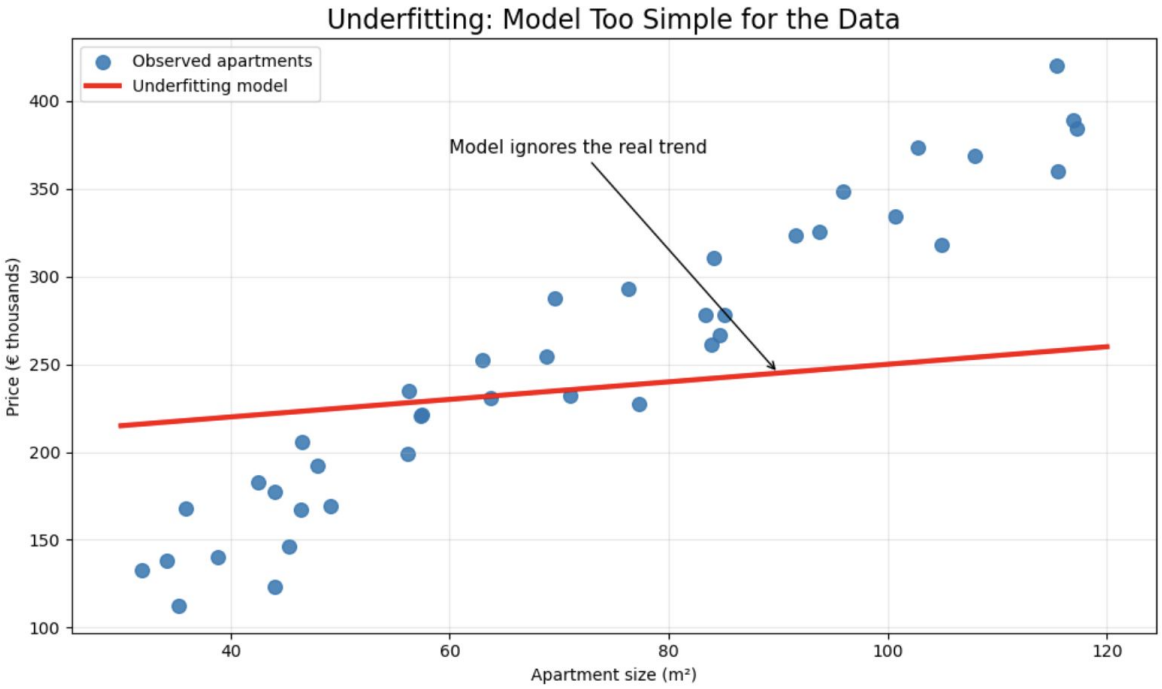
Overfitting

Connecting all points memorizes the data, but does not capture the pattern.



Underfitting

Model ignores the data.



Train/Test split

🔵 Training set

- used to learn the model
- adjust weights
- seen many times

🟣 Dev set (Validation set)

- used to tune the model
- choose hyperparameters
- compare different models

🟠 Test set

- used to evaluate the final model
- never used during training or tuning

⚠️

❌ Never test on training data
❌ Never tune on test data
❌ Avoid data leakage
❌ Don’t trust one metric blindly

Data leakage = using information that would not be available in real life
→ leads to unrealistic performance

What’s a good training?

Look at the loss.

Mini Assignment

Clone the repo!

TODOs:

- Exercise notebook in repo